Smart Campus Incident and Maintenance Prioritization

Python Programming – Activity-Based Learning (ABL) Assignment

1. Problem Statement

Classify hypothetical maintenance reports using severity, location, and reported frequency; rank cases using predefined rules and generate a priority summary.

2. Objectives

- Store maintenance incidents.
- Calculate a priority score from severity, location, and frequency.
- Classify each report as Critical, High, Medium, or Low.
- Rank reports and summarize priority counts.

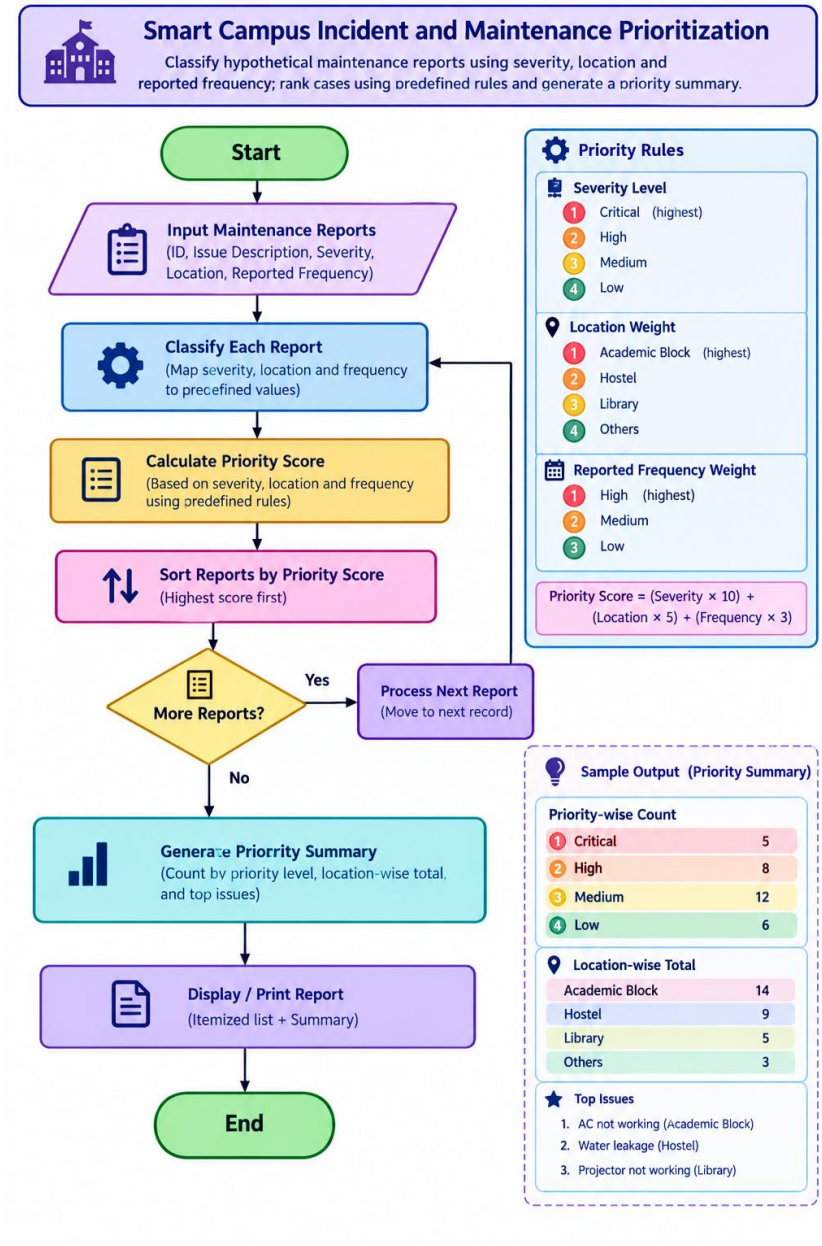
3. Assumptions and Priority Policy

Severity uses a 1–5 scale, where 5 is safety-critical. Frequency uses a 1–5 scale, where 5 means repeatedly reported. The sample score is severity × 3 + location weight + frequency × 2. Severity 5 or score at least 18 is Critical; score 13–17 is High; score 8–12 is Medium; otherwise Low. Location weights are illustrative and can be adjusted to campus policy. These reports are hypothetical.

4. Algorithm

1. Store incident ID, description, severity, location, and frequency.
2. Look up the location weight.
3. Calculate score = severity*3 + location weight + frequency*2.
4. Assign priority using the predefined thresholds.
5. Sort by score descending, then severity and frequency.
6. Print the ranked incident list and counts by priority.

5. Flowchart



6. Pseudocode

BEGIN

DECLARE reports as empty list

```
INPUT number_of_reports

FOR i = 1 TO number_of_reports DO

  INPUT report_id
  INPUT issue_description
  INPUT severity
  INPUT location
  INPUT frequency

  // Assign Severity Weight
  IF severity = "Critical" THEN
    severity_weight = 4
  ELSE IF severity = "High" THEN
    severity_weight = 3
  ELSE IF severity = "Medium" THEN
    severity_weight = 2
  ELSE
    severity_weight = 1
  END IF

  // Assign Location Weight
  IF location = "Academic Block" THEN
    location_weight = 4
  ELSE IF location = "Hostel" THEN
```

```
    location_weight = 3
ELSE IF location = "Library" THEN
    location_weight = 2
ELSE
    location_weight = 1
END IF

// Assign Frequency Weight
IF frequency = "High" THEN
    frequency_weight = 3
ELSE IF frequency = "Medium" THEN
    frequency_weight = 2
ELSE
    frequency_weight = 1
END IF

// Calculate Priority Score
priority_score =
    (severity_weight * 10) +
    (location_weight * 5) +
    (frequency_weight * 3)

// Classify Priority
IF priority_score >= 55 THEN
    priority = "Critical"
ELSE IF priority_score >= 40 THEN
```

```
    priority = "High"
  ELSE IF priority_score >= 25 THEN
    priority = "Medium"
  ELSE
    priority = "Low"
  END IF

  ADD report details, priority_score,
    and priority TO reports

END FOR

// Sort reports by priority score
SORT reports BY priority_score IN DESCENDING ORDER

// Generate Priority Summary
COUNT Critical reports
COUNT High reports
COUNT Medium reports
COUNT Low reports

DISPLAY "SMART CAMPUS PRIORITY REPORT"

FOR each report IN reports DO
  DISPLAY report_id
  DISPLAY issue_description
```

```
        DISPLAY severity
        DISPLAY location
        DISPLAY frequency
        DISPLAY priority_score
        DISPLAY priority
    END FOR

    DISPLAY "Priority-wise Summary"
    DISPLAY Critical report count
    DISPLAY High report count
    DISPLAY Medium report count
    DISPLAY Low report count

    DISPLAY "Reports ranked by highest priority first"

END
```

7. Python Source Code

```
def get_severity_weight(severity):
        weights = {
              "critical": 4,
               "high": 3,
               "medium": 2,
               "low": 1
 }
 return weights.get(severity.lower(), 1)
```

```
def get_location_weight(location):
      weights = {
            "academic block": 4,
             "hostel": 3,
             "library": 2,
             "others": 1
 }
  return weights.get(location.lower(), 1)
def get_frequency_weight(frequency):
       weights = {
              "high": 3,
               "medium": 2,
               "low": 1
}
   return weights.get(frequency.lower(), 1)
def classify_priority(score):
       if score >= 55:
              return "Critical"
       elif score >= 40:
              return "High"
       elif score >= 25:
              return "Medium"
```

```
    else:
        return "Low"

def main():
print("=" * 75)
print(" SMART CAMPUS INCIDENT AND MAINTENANCE PRIORITIZATION")
print("=" * 75)
reports = []

n = int(input("Enter number of maintenance reports: "))

for i in range(n):
    print(f"\nEnter details for Report {i + 1}")

    report_id = input("Report ID: ")
    issue = input("Issue description: ")

    severity = input(
        "Severity (Critical/High/Medium/Low): "
    ).strip()

    location = input(
        "Location (Academic Block/Hostel/Library/Others): "
    ).strip()

    frequency = input(
        "Reported frequency (High/Medium/Low): "
    ).strip()

    severity_weight = get_severity_weight(severity)
    location_weight = get_location_weight(location)
    frequency_weight = get_frequency_weight(frequency)

    # Calculate priority score
    score = (
```

```
        severity_weight * 10
        + location_weight * 5
        + frequency_weight * 3
    )

    priority = classify_priority(score)

    report = {
        "id": report_id,
        "issue": issue,
        "severity": severity,
        "location": location,
        "frequency": frequency,
        "score": score,
        "priority": priority
    }

    reports.append(report)

    print("Report added successfully!")

# Sort reports by score, highest first
reports.sort(key=lambda r: r["score"], reverse=True)

# Generate priority summary
summary = {
    "Critical": 0,
    "High": 0,
    "Medium": 0,
    "Low": 0
}

location_summary = {}

for report in reports:
    summary[report["priority"]] += 1

    location = report["location"].title()
    location_summary[location] = (
```

```
        location_summary.get(location, 0) + 1
    )

# Display ranked reports
print("\n" + "=" * 100)
print("                 RANKED MAINTENANCE REPORTS")
print("=" * 100)

print(
    f"{'Rank':<6}{'ID':<10}{'Issue':<24}"
    f"{'Location':<18}{'Score':<10}{'Priority'}"
)
print("-" * 100)

for rank, report in enumerate(reports, start=1):
    print(
        f"{rank:<6}{report['id']:<10}"
        f"{report['issue'][:22]:<24}"
        f"{report['location'][:16]:<18}"
        f"{report['score']:<10}"
        f"{report['priority']}"
    )

# Display complete report details
print("\n" + "=" * 75)
print("                 PRIORITY SUMMARY")
print("=" * 75)

for priority, count in summary.items():
    print(f"{priority:<15}: {count}")

print("\nLocation-wise Report Count:")
for location, count in location_summary.items():
    print(f"{location:<20}: {count}")

if reports:
    print("\nHighest Priority Issue:")
    top = reports[0]
    print(f"Report ID   : {top['id']}")
```

```
    print(f"Issue       : {top['issue']}")
    print(f"Location    : {top['location']}")
    print(f"Priority    : {top['priority']}")
    print(f"Score       : {top['score']}")

print("\n" + "=" * 75)
print("          MAINTENANCE PRIORITIZATION COMPLETED")
print("=" * 75)
```

if **name** == "**main**": main()

8. Output Screenshot

```
===========================================================================
 SMART CAMPUS INCIDENT AND MAINTENANCE PRIORITIZATION
===========================================================================
Enter number of maintenance reports: 3

Enter details for Report 1
Report ID: R101
Issue description: AC not working
Severity (Critical/High/Medium/Low): Critical
Location (Academic Block/Hostel/Library/Others): Block
Reported frequency (High/Medium/Low): High
Report added successfully!

Enter details for Report 2
Report ID: R102
Issue description: Water leakage
Severity (Critical/High/Medium/Low): High
Location (Academic Block/Hostel/Library/Others): Hostel
Reported frequency (High/Medium/Low): Medium
Report added successfully!

Enter details for Report 3
Report ID: R103
Issue description: Projector issue
Severity (Critical/High/Medium/Low): Medium
Location (Academic Block/Hostel/Library/Others): Library
Reported frequency (High/Medium/Low): Low
Report added successfully!
```

```
================================================================================
============
                RANKED MAINTENANCE REPORTS
================================================================================
============
Rank  ID        Issue                  Location          Score     Priority
--------------------------------------------------------------------------------
------------
1     R101      AC not working         Block             54        High
2     R102      Water leakage          Hostel            51        High
3     R103      Projector issue        Library           33        Medium

================================================================================
                 PRIORITY SUMMARY
================================================================================
Critical       : 0
High           : 2
Medium         : 1
Low            : 0

Location-wise Report Count:
Block               : 1
Hostel              : 1
Library             : 1

Highest Priority Issue:
Report ID   : R101
Issue       : AC not working
Location    : Block
Priority    : High
Score       : 54

================================================================================
          MAINTENANCE PRIORITIZATION COMPLETED
================================================================================
```

9. Test Cases

1. Severity 5 report -> classified Critical under the example policy.

2. Larger reported frequency -> adds 2 points per frequency level.

3. Main Lab / Hostel Block -> location weight 3 in this sample.

4. Reports are ranked by score descending.

5. Equal scores are resolved by severity, frequency, then incident ID.

6. Priority summary counts incidents in each category.

10. Conclusion

The program calculates scores from severity, location, and frequency, classifies and ranks hypothetical maintenance reports, and prints a summary to help staff identify urgent cases. Real campus safety procedures should override any illustrative score.